Attention Is All You Need

Ashish Vaswani[*]
Google Brain
avaswani@google.com

Noam Shazeer[*]
Google Brain
noam@google.com

Niki Parmar[*]
Google Research
nikip@google.com

Jakob Uszkoreit[*]
Google Research
usz@google.com

Llion Jones[*]
Google Research
llion@google.com

Aidan N. Gomez[*] [†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser[*]
Google Brain
lukaszkaiser@google.com

Illia Polosukhin[*] [‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

1 Introduction

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [29, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [31, 21, 13].

[*]Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Attention Is All You Need

Ashish Vaswani
Google Brain
avaswani@google.com

Noam Shazeer
Google Brain
noam@google.com

Niki Parmar
Google Research
nikip@google.com

Jakob Uszkoreit
Google Research
usz@google.com

Llion Jones
Google Research
llion@google.com

Aidan N. Gomez
多伦多大学
aidan@cs.toronto.edu

卢卡什・凯瑟
Google Brain
lukaszkaiser@google.com

伊利亚・波洛苏欣
illia.polosukhin@gmail.com

摘要

目前主流的序列转换模型基于复杂的循环或卷积神经网络，这些网络包含编码器和解码器。表现最好的模型也通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构，即 Transformer，该架构仅基于注意力机制，完全摒弃了循环和卷积。我们在两个机器翻译任务上的实验表明，这些模型在质量上更优，同时具备更好的并行化能力，并且训练所需时间显著减少。我们的模型在 WMT 2014 英语到德语翻译任务上取得了 28.4 BLEU 分数，比现有最佳结果（包括集成模型）提高了超过 2 BLEU。在 WMT 2014 英语到法语翻译任务上，我们的模型在八块 GPU 上训练了 3.5 天后，建立了新的单模型最先进水平的 41.0 BLEU 分数，这仅是文献中最佳模型训练成本的一小部分。

1 引言

循环神经网络，特别是长短期记忆[12]和门控循环[7]神经网络，在序列建模和转换问题如语言模型和机器翻译[29, 2, 5]中已被确立为最先进的方法。此后，许多研究继续致力于推动循环语言模型和编码器-解码器架构的边界[31, 21, 13]。

同等贡献。排序是随机的。Jakob 提出用 self-attention 替代 RNN，并启动了评估这一想法的工作。Ashish 与 Illia 一起设计并实现了第一个 Transformer 模型，并在本工作的各个方面都起到了关键作用。Noam 提出了 scaled dot-product attention、multi-head attention 以及无参数的位置表示，并成为几乎每个细节的参与者。Niki 在我们最初代码库和 tensor2tensor 中设计、实现、调整和评估了无数的模型变体。Llion 也尝试了新的模型变体，负责我们最初的代码库，并实现了高效的推理和可视化。Lukasz 和 Aidan 花费了无数个漫长的日夜来设计和实现 tensor2tensor 的各个部分，替换了我们之前的代码库，显著提升了结果，并极大地加速了我们的研究。

[†]在 Google Brain 工作期间完成的工作。
[‡]在 Google Research 工作期间完成的工作。

第 31 届神经信息处理系统大会（NIPS 2017），美国加利福尼亚州长滩。

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [18] and conditional computation [26], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 16]. In all but a few cases [22], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [20], ByteNet [15] and ConvS2S [8], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [11]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 22, 23, 19].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [28].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [14, 15] and [8].

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 29]. Here, the encoder maps an input sequence of symbol representations $(x_1, ..., x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, ..., z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, ..., y_m)$ of symbols one element at a time. At each step the model is auto-regressive [9], consuming the previously generated symbols as additional input when generating the next.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-

循环模型通常沿输入和输出序列的符号位置进行计算分解。将位置对齐到计算步骤中，它们生成一个隐藏状态序列 h，该序列作为前一个隐藏状态 h_{t-1}和位置 t 的输入的函数。这种本质上序列化的特性使得在训练样本内部无法并行化，这在序列长度较长时变得尤为关键，因为内存限制会阻碍跨样本的批量处理。近期的工作通过分解技巧[18]和条件计算[26]显著提高了计算效率，同时在后一种情况下也改善了模型性能。然而，顺序计算的基本限制仍然存在。

注意力机制已成为各种任务中引人注目的序列建模和转换模型的重要组成部分，使得无需考虑输入或输出序列中依赖关系的距离即可进行建模[2, 16]。然而，在几乎所有情况下[22]，这种注意力机制都是与循环网络结合使用的。

在本工作中，我们提出了 Transformer，这是一种摒弃循环结构，完全依赖注意力机制来建立输入和输出之间全局依赖关系的模型架构。Transformer 允许显著更多的并行化处理，并且在仅使用八块 P100 GPU 训练十二小时后，就能在翻译质量上达到新的领先水平。

2　背景

减少序列计算的目标也构成了扩展神经 GPU [20]、ByteNet [15] 和 ConvS2S [8] 的基础，这些模型都使用卷积神经网络作为基本构建模块，能够并行计算所有输入和输出位置的隐藏表示。在这些模型中，将两个任意输入或输出位置之间的信号联系所需的计算操作数量随着位置间的距离增加而增长，ConvS2S 是线性增长，ByteNet 是对数增长。这使得学习远距离位置之间的依赖关系变得更加困难[11]。在 Transformer 中，这一操作数量被减少到一个常数，尽管由于对注意力加权位置进行平均，导致有效分辨率降低，我们通过第 3.2 节中描述的多头注意力机制来抵消这一影响。

自注意力，有时也称为内部注意力，是一种将单个序列中不同位置关联起来以计算序列表示的注意力机制。自注意力已被成功应用于多种任务，包括阅读理解、摘要生成、文本蕴含以及学习与任务无关的句子表示 [4, 22, 23, 19]。

端到端记忆网络基于循环注意力机制，而不是序列对齐的循环结构，并且在简单语言的问题回答和语言建模任务中表现出色 [28]。

据我们所知，Transformer 是首个完全依赖自注意力来计算其输入和输出表示的转换模型，而无需使用序列对齐的 RNN 或卷积。在接下来的章节中，我们将描述 Transformer，阐述自注意力的动机，并讨论其相较于 [14, 15] 和 [8] 等模型的优势。

3　模型架构

大多数表现优异的神经序列转换模型都采用编码器-解码器结构 [5, 2, 29]。在这里，编码器将输入序列的符号表示（x, ..., x）映射为连续表示的序列 z = (z, ..., z)。给定 z，解码器随后逐个元素生成输出序列（y, ..., y）的符号。在每一步中，模型都是自回归的 [9]，在生成下一个符号时，会将之前生成的符号作为额外输入。

Transformer 模型遵循这种总体架构，使用堆叠的自注意力机制和点-wise、全连接层作为编码器和解码器，如图 1 的左右两部分所示。

3.1　编码器和解码器堆栈

编码器：编码器由 N = 6 个相同的层堆叠而成。每个层包含两个子层。第一个是多头自注意力机制，第二个是一个简单的、基于位置的

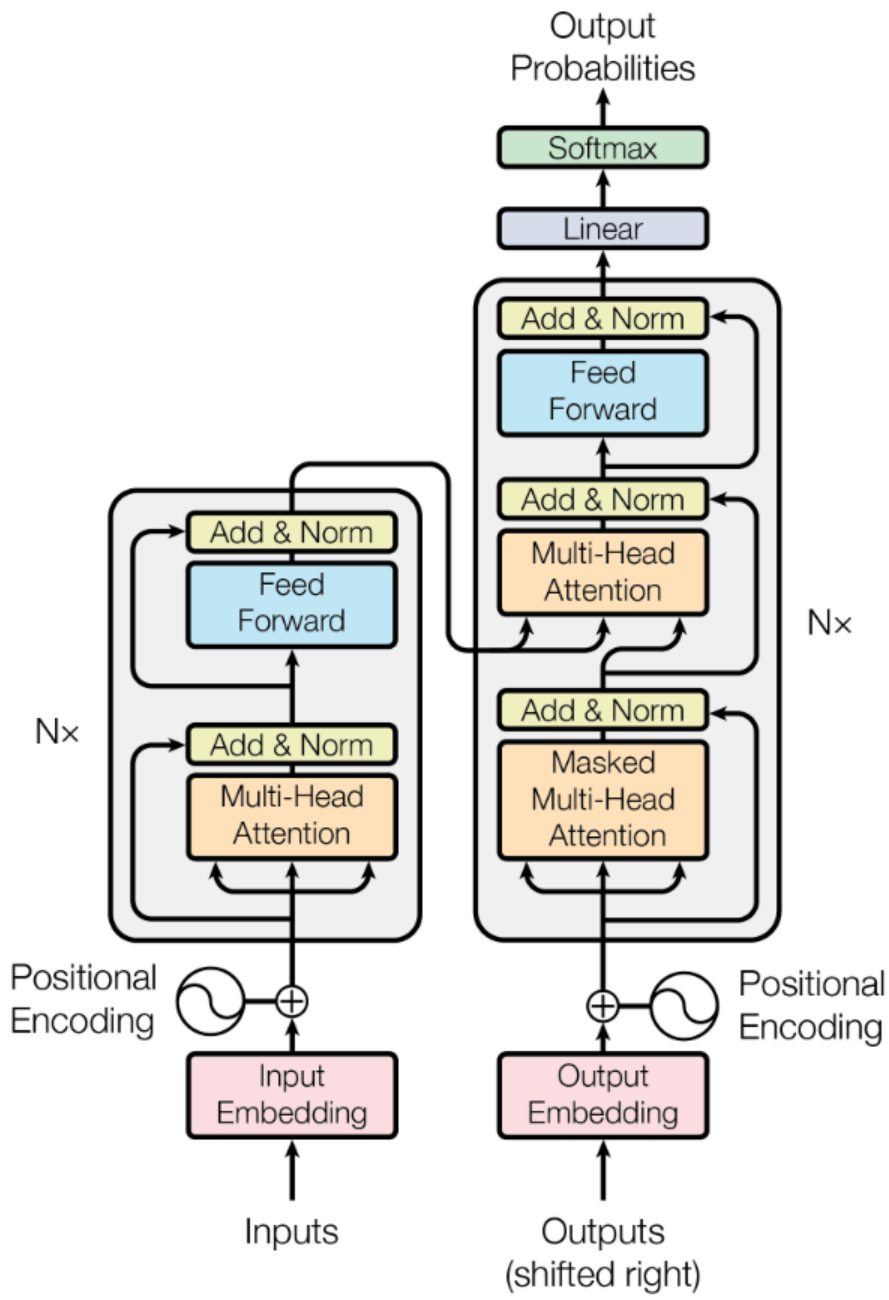


Figure 1: The Transformer - model architecture.

wise fully connected feed-forward network. We employ a residual connection [10] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$, where $\mathrm{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the

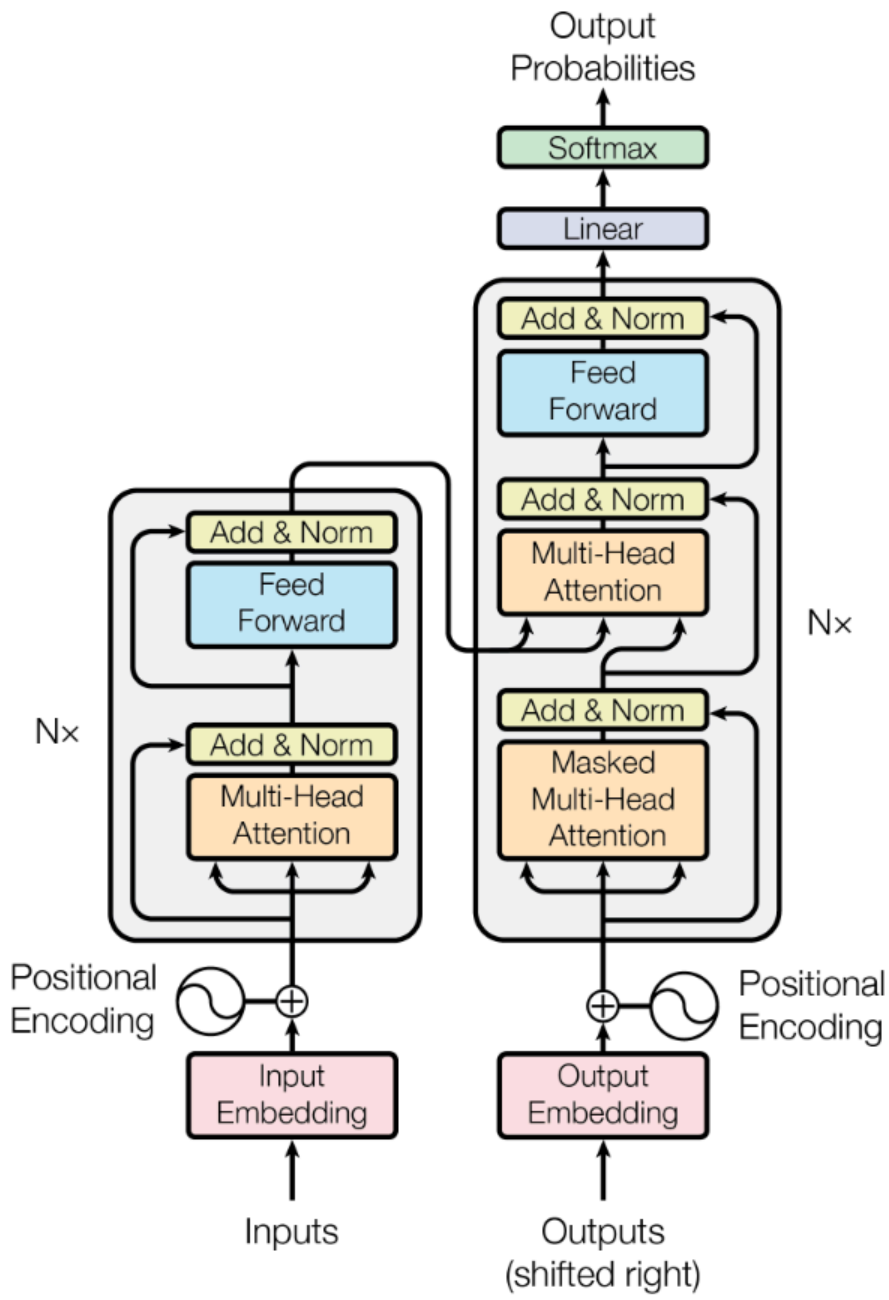


图 1：Transformer 模型结构。

每一层包含两个子层，每个子层都是全连接的前馈网络。我们在每个子层周围使用残差连接[10]，然后进行层归一化[1]。也就是说，每个子层的输出为 LayerNorm(x + Sublayer(x))，其中 Sublayer(x) 是该子层自身实现的函数。为了便于进行这些残差连接，模型中的所有子层以及嵌入层都产生维度为 d=512 的输出。

解码器：解码器也由 N = 6 个相同的层堆叠而成。除了每个编码器层中的两个子层外，解码器还插入了一个第三个子层，该子层在编码器堆叠的输出上执行多头注意力。与编码器类似，我们在每个子层周围使用残差连接，然后进行层归一化。我们还修改了解码器堆叠中的自注意力子层，以防止位置关注后续的位置。这种掩码，结合输出嵌入层在位置上偏移一位的事实，确保了位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2　注意力

注意力函数可以描述为将一个查询和一组键-值对映射到输出，其中查询、键、值和输出都是向量。输出是值的加权和，其中每个值的权重由查询与相应键的兼容性函数计算得出。

3.2.1　缩放点积注意力

我们称我们的特定注意力为“缩放点积注意力”（图 2）。输入包括维度为 d 的查询和键，以及维度为 d 的值。我们计算查询与键的点积，

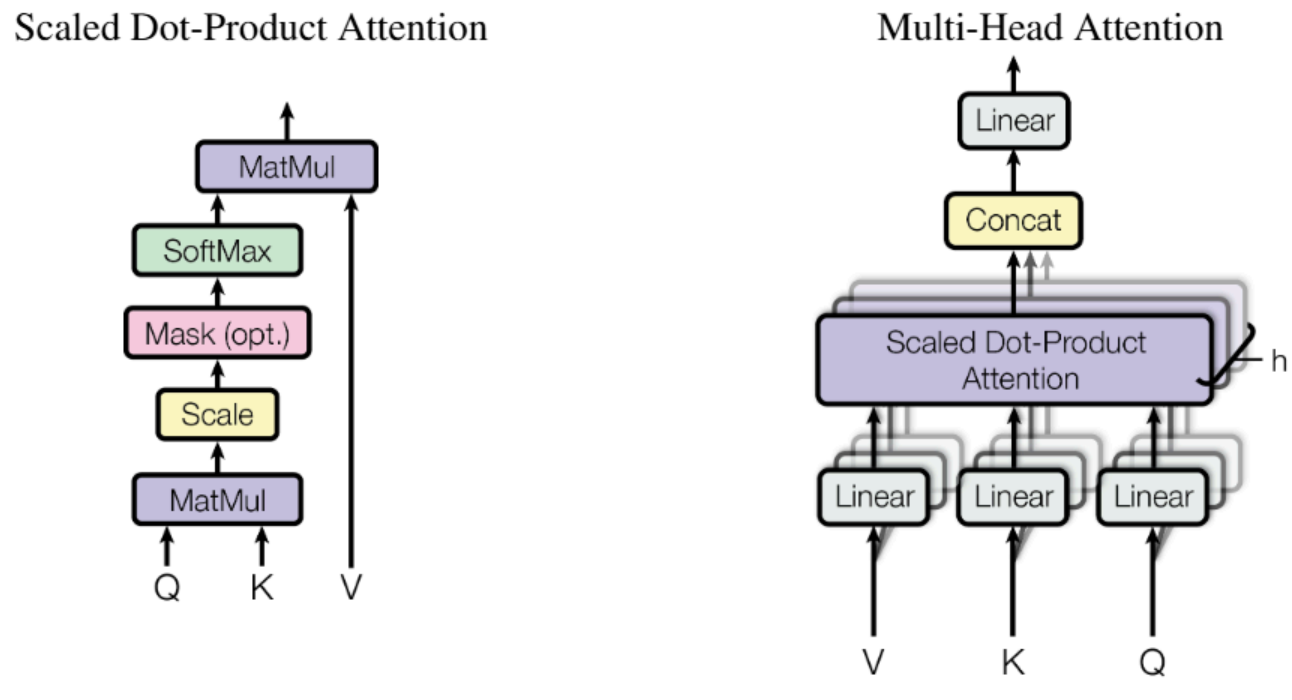


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients [4]. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

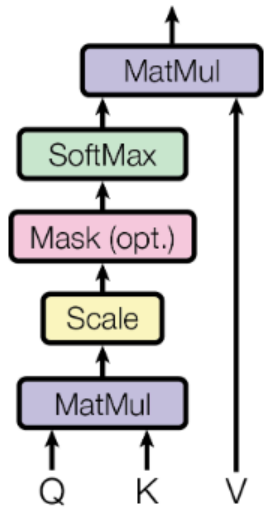
3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model}-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

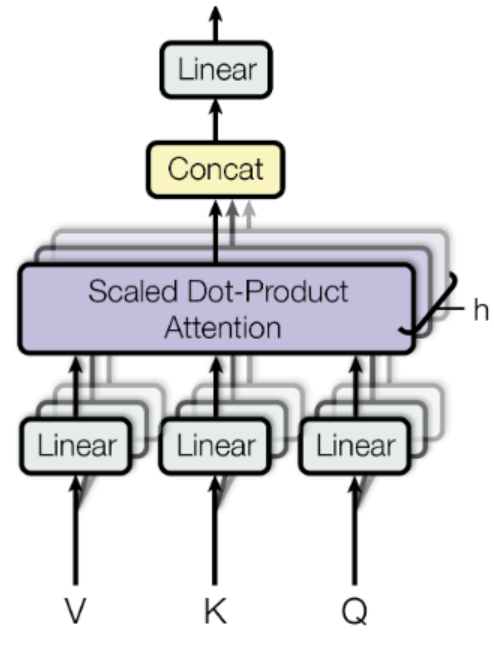
Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

[4] To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k.

缩放点积注意力



多头注意力



（图 2）图 2：（左）缩放点积注意力。 （右）多头注意力由几个并行运行的注意力层组成。

用所有键进行查询，每个结果都除以$\sqrt{d}$，并对其应用 softmax 函数以获得值上的权重。
值。

实际上，我们同时计算一组查询的注意力函数，将这些查询打包成一个矩阵 Q。键和值也被打包成矩阵 K 和 V。我们计算输出矩阵如下：

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK}{\sqrt{d}})V \tag{1}$$

目前最常用的两种注意力机制是加法注意力[2]和点积（乘法）注意力。点积注意力与我们的算法相同，只是在点积上乘以一个缩放因子。加法注意力使用一个具有单个隐藏层的前馈网络来计算兼容性函数。虽然从理论复杂度来看两者相似，但在实践中，点积注意力要快得多且空间效率更高，因为它可以使用高度优化的矩阵乘法代码来实现。

尽管对于较小的 d 值，这两种机制表现相似，但不进行缩放的点积注意力在较大的 d 值下表现不如加法注意力[3]。我们怀疑，对于较大的 d 值，点积的绝对值会变得很大，从而将 softmax 函数推向梯度极其微小的区域。为了抵消这种影响，我们对点积进行了缩放。

3.2.2 多头注意力

我们发现，通过使用不同的、可学习的线性投影，将查询、键和值分别线性投影到 d 维、d 维和 d 维，然后在这些投影后的查询、键和值上并行执行注意力函数，而不是仅使用一个 d 维的注意力函数，这种方法更有益。这样，每个投影后的查询、键和值都会产生 d 维的输出值。这些输出值被连接起来，再次进行投影，从而得到最终的输出值，如图 2 所示。

多头注意力允许模型在不同位置上同时关注不同表示子空间中的信息。使用单个注意力头时，平均值会抑制这种效果。

为了说明为什么点积会变得很大，假设 q 和 k 的各个分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积，$q \cdot k = \sum_{i=1}^{d} qk$，其均值为 0，方差为 d。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_\text{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_\text{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_\text{model} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_\text{model}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_\text{model}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [31, 2, 8].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_\text{model} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_model. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [24]. In the embedding layers, we multiply those weights by $\sqrt{d_\text{model}}$.

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}, ..., \text{head})W$$

$$\text{其中 head} = \text{Attention}(QW, KW, VW)$$

其中投影是参数矩阵 $W \in R$, $W \in R$, $W \in R$ 和 $W \in R$。

在本工作中，我们采用 h = 8 个并行的注意力层，或称为头。对于每个头，我们使用 d= d= d/h = 64。由于每个头的维度被降低，总的计算成本与全维度单头注意力机制的计算成本相当。

3.2.3 注意力在我们模型中的应用

Transformer 在三个方面使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自前一层解码器，记忆的键和值来自编码器的输出。这使得解码器中的每个位置都能关注输入序列中的所有位置。这模仿了序列到序列模型中典型的编码器-解码器注意力机制，例如 [31, 2, 8]。

- 编码器包含自注意力层。在自注意力层中，所有的键、值和查询都来自同一个位置，此处指的是编码器中前一层的输出。编码器中的每个位置都可以关注编码器前一层中的所有位置。

- 同样地，解码器中的自注意力层允许解码器中的每个位置关注解码器中包括该位置在内的所有先前位置。我们需要在解码器中防止向左的信息流动，以保持自回归属性。我们通过在缩放点积注意力中屏蔽掉（设置为−∞）softmax 输入中对应非法连接的所有值来实现这一点。见图 2。

3.3 位置级前馈网络

除了注意力子层，我们的编码器和解码器中的每一层还包含一个全连接的前馈网络，该网络对每个位置独立且相同地应用。这由两个线性变换组成，在中间使用 ReLU 激活函数。

$$\text{FFN}(x) = \max(0, xW + b)W + b \tag{2}$$

虽然不同位置的线性变换是相同的，但它们在层之间使用不同的参数。另一种描述方式是两个卷积层，其卷积核大小为 1。输入和输出的维度是 d= 512，中间层的维度是 d= 2048。

3.4 Embeddings and Softmax

同样地，与其他序列转换模型类似，我们使用学习得到的嵌入层将输入标记和输出标记转换为维度为 d 的向量。我们还使用常规的学习得到的线性变换和 softmax 函数将解码器的输出转换为预测的下一个标记的概率。在我们的模型中，我们将在两个嵌入层和预 softmax 线性变换之间共享相同的权重矩阵，类似于[24]。在嵌入层中，我们乘以这些权重。 $\sqrt{d}$.

3.5 位置编码

由于我们的模型不包含循环和卷积结构，为了让模型能够利用序列中的顺序信息，我们必须向输入嵌入中注入一些关于序列中令牌的相对或绝对位置的信息。为此，我们在输入嵌入中添加了“位置编码”。

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [8].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

We also experimented with using learned positional embeddings [8] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, ..., x_n)$ to another sequence of equal length $(z_1, ..., z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [11]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [31] and byte-pair [25] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in

为此，表 1：不同层类型的最长路径长度、每层复杂度和最少的顺序操作数。n 是序列长度，d 是表示维度，k 是卷积的核大小，r 是受限自注意力中的邻域大小。

层类型	每层复杂度	顺序操作	最大路径长度
自注意力	O(n · d)	O(1)	O(1)
循环	O(n · d)	O(n)	O(n)
卷积	O(k · n · d)	O(1)	O(log(n))
自注意力（受限）	O(r · n · d)	O(1)	O(n/r)

编码器和解码器堆栈的底部。位置编码的维度与嵌入相同，因此两者可以相加。有很多位置编码的选择，包括学习的和固定的[8]。

在本工作中，我们使用不同频率的正弦和余弦函数：

$$PE = \sin(pos/10000)$$

$$PE = \cos(pos/10000)$$

其中 pos 是位置，i 是维度。也就是说，每个位置编码的维度对应一个正弦波。这些正弦波的波长从 2π 到 10000 · 2π 形成一个等比数列。我们选择这个函数是因为我们假设它将使模型能够轻松地通过相对位置来学习注意力机制，因为对于任何固定的偏移量 k，PE 可以表示为 PE 的线性函数。

我们还尝试使用学习得到的位置嵌入 [8]，并发现两种方法产生的结果几乎相同（见表 3 行 (E)）。我们选择正弦版本是因为它可能允许模型外推到训练过程中未遇到的更长序列长度。

4 为什么使用自注意力

在本节中，我们将自注意力层与常用于将一个可变长度的符号表示序列（x, ..., x）映射到另一个等长序列（z, ..., z）的循环层和卷积层进行比较，其中 x, z∈ R，例如典型序列转换编码器或解码器中的隐藏层。我们使用自注意力的动机基于三个期望特性。

每一层的总计算复杂度是一个因素。另一个因素是能够并行计算的计算量，这可以通过所需的最少顺序操作数来衡量。

第三个因素是网络中长距离依赖关系之间的路径长度。学习长距离依赖关系是许多序列转换任务中的关键挑战。影响学习这种依赖关系能力的一个关键因素是信号在网络中向前和向后传播时需要经过的路径长度。输入和输出序列中任意两个位置之间的路径越短，学习长距离依赖关系就越容易[11]。因此，我们还比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表 1 所示，自注意力层通过一个常数数量的顺序执行操作连接所有位置，而循环层则需要 O(n)的顺序操作。在计算复杂度方面，当序列长度 n 小于表示维度 d 时，自注意力层比循环层更快，这种情况在大多数机器翻译中使用句子表示的最先进模型中最为常见，例如 word-piece [31]和 byte-pair [25]表示方法。为了提高处理非常长序列任务的计算性能，自注意力可以被限制仅考虑大小为 r 的局部区域。

the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(log_k(n))$ in the case of dilated convolutions [15], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k. Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

5 Training

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [31]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

5.3 Optimizer

We used the Adam optimizer [17] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \tag{3}$$

This corresponds to increasing the learning rate linearly for the first $warmup_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup_steps = 4000$.

5.4 Regularization

We employ three types of regularization during training:

Residual Dropout We apply dropout [27] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

这在大多数情况下都适用于机器翻译中最新模型使用的句子表示，即输入序列围绕相应的输出位置进行中心化。这将使最大路径长度增加到 O(n/r)。我们计划在今后的工作中进一步研究这种方法。

一个单一的卷积层，其核宽度 k < n，并不能连接所有输入和输出位置的配对。要实现这一点，在连续核的情况下需要堆叠 O(n/k)个卷积层，而在扩张卷积的情况下则需要 O(log(n))个卷积层[15]，从而增加网络中任意两个位置之间的最长路径长度。通常来说，卷积层的计算成本比循环层高，高出 k 倍。然而，可分离卷积[6]显著降低了复杂度，降至 O(k · n · d + n · d)。即使在 k = n 的情况下，可分离卷积的复杂度也等同于自注意力层和点-wise 前馈层的组合，这也是我们模型所采用的方法。

作为额外的好处，自注意力机制可能产生更易解释的模型。我们检查了模型中的注意力分布，并在附录中展示了并讨论了相关示例。不仅每个注意力头都明显地学习执行不同的任务，许多注意力头似乎表现出与句子的句法和语义结构相关的行为。

5 训练

本节描述了我们模型的训练方式。

5.1 训练数据和批处理

我们在标准的 WMT 2014 英语-德语数据集上进行训练，该数据集包含大约 450 万句对。句子使用字节对编码 [3] 进行编码，其共享的源-目标词汇表包含约 37000 个词符。对于英语-法语，我们使用了显著更大的 WMT 2014 英语-法语数据集，包含 3600 万句，并将词符拆分为一个 32000 词的词片词汇表 [31]。句对按照近似序列长度进行批处理。每个训练批次包含一组句对，大约包含 25000 个源词符和 25000 个目标词符。

5.2 硬件和训练计划

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练我们的模型。对于我们在论文中描述的基线模型，每个训练步骤大约需要 0.4 秒。我们对基线模型进行了总共 100000 个步骤或 12 小时的训练。对于我们的大模型（见表 3 最后一行），每个步骤耗时 1.0 秒。大模型的训练共进行了 300000 个步骤（约 3.5 天）。

5.3 Optimizer

我们使用了 Adam 优化器 [17]，其参数为 β= 0.9， β= 0.98 和 ☒ = 10。我们在训练过程中根据以下公式调整学习率：

$$lrate = dmodel \cdot min(step_num, step_num \cdot warmup_steps) \quad (3)$$

这表示在前 warmup_steps 个训练步骤中，学习率线性增加，之后则按步数的倒数平方根比例递减。我们使用了 warmup_steps = 4000。

5.4 正则化

我们在训练过程中采用三种类型的正则化：

残差丢弃（Residual Dropout）我们在每个子层的输出上应用丢弃（dropout）[27]，在将其加到子层输入并进行归一化之前。此外，我们在编码器和解码器堆栈中对嵌入和位置编码的和也应用丢弃。对于基础模型，我们使用丢弃率 P= 0.1。

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.0**	$2.3 \cdot 10^{19}$	

Label Smoothing During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [30]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Results

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than $1/4$ the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [31]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [31].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU [5].

6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

[5] We used values of 2.8, 3.7, 6.0 and 9.5 TFLOPS for K80, K40, M40 and P100, respectively.

表 2：在英语到德语和英语到法语的 newstest2014 测试中，Transformer 模型在较少的训练成本下取得了比之前最先进的模型更好的 BLEU 分数。

模型	BLEU		训练成本（FLOPs）	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		1.0 · 10
GNMT + RL [31]	24.6	39.92		2.3 · 101.4 · 10
ConvS2S [8]	25.16	40.46	9.6 · 101.5 · 10	
MoE [26]	26.03	40.56		2.0 · 101.2 · 10
Deep-Att + PosUnk Ensemble [32]		40.4		8.0 · 10
GNMT + RL Ensemble [31]	26.30	41.16		1.8 · 101.1 · 10
ConvS2S Ensemble [8]	26.36	41.29	7.7 · 101.2 · 10	
Transformer (base model)	27.3	38.1	3.3 · 10	
Transformer (big)	28.4	41.0	2.3 · 10	

训练过程中，我们采用了值为⊠= 0.1 的标签平滑 [30]。这会降低困惑度，因为模型学习得更加不确定，但可以提高准确率和 BLEU 分数。

6 结果

6.1 机器翻译

在 WMT 2014 英语到德语的翻译任务中，大型 transformer 模型（表 2 中的 Transformer (big)）超越了之前所有报告的最佳模型（包括集成模型），提升了超过 2.0 个 BLEU 分数，建立了新的 BLEU 分数记录，达到 28.4 分。该模型的配置列于表 3 的最后一行。训练该模型使用了 8 块 P100 GPU，耗时 3.5 天。即使我们的基础模型也超越了之前所有发表的模型和集成模型，其训练成本仅为竞争模型之一的四分之一左右。

在 WMT 2014 英语到法语的翻译任务中，我们的大型模型取得了 41.0 的 BLEU 分数，超越了之前所有发表的单模型，其训练成本仅为之前最佳模型的四分之一不到。用于英语到法语的 Transformer (big)模型采用了丢弃率 P=0.1，而不是 0.3。

对于基础模型，我们使用了通过平均最后 5 个检查点得到的单个模型，这些检查点每隔 10 分钟保存一次。对于较大的模型，我们平均了最后 20 个检查点。我们使用了带有束宽为 4 和长度惩罚因子α = 0.6 的束搜索 [31]。这些超参数是在开发集上的实验后选择的。我们在推理时将最大输出长度设置为输入长度加 50，但可能提前终止 [31]。

表 2 总结了我们的结果，并将我们的翻译质量与训练成本与其他模型架构进行了比较。我们通过将训练时间、使用的 GPU 数量以及每块 GPU 的持续单精度浮点运算能力估计值相乘，来估算训练模型所使用的浮点运算次数。

6.2 模型变体

为了评估 Transformer 中不同组件的重要性，我们以不同的方式修改了基础模型，并在开发集 newstest2013 上测量了其在英语到德语翻译任务中的性能变化。我们使用了前一节中描述的束搜索，但没有使用检查点平均。我们在表 3 中展示了这些结果。

在表 3 的行(A)中，我们变化注意力头的数量以及注意力键和值的维度，同时保持计算量不变，如第 3.2.2 节所述。虽然单头注意力比最佳设置低 0.9 BLEU，但注意力头过多也会导致质量下降。

[5] 我们分别使用了 2.8、3.7、6.0 和 9.5 TFLOPS 的值用于 K80、K40、M40 和 P100。

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [8], and observe nearly identical results to the base model.

7 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

The code we used to train and evaluate our models is available at `https://github.com/tensorflow/tensor2tensor`.

Acknowledgements We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

表 3：Transformer 架构的变体。未列出的值与基础模型相同。所有指标均基于英语到德语的翻译开发集 newstest2013。列出的困惑度是按词元计算的，根据我们的字节对编码，不应与按单词计算的困惑度进行比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)						16				5.16	25.1	58
						32				5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding 代替 sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

在表 3 的行(B)中，我们观察到减少注意力键的大小 d_k 模型质量。这表明确定兼容性并不容易，可能需要比点积更复杂的兼容性函数。我们进一步在行(C)和(D)中观察到，如预期那样，更大的模型表现更好，而 dropout 在避免过拟合方面非常有用。在行(E)中，我们将我们的正弦位置编码替换为学习得到的位置嵌入[8]，并观察到与基础模型几乎相同的结果。

7 结论

在这项工作中，我们提出了 Transformer，这是首个完全基于注意力的序列转换模型，用多头自注意力替换了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度显著快于基于循环或卷积层的架构。在 WMT 2014 英语到德语和 WMT 2014 英语到法语的翻译任务中，我们取得了新的最先进水平。在前一项任务中，我们的最佳模型甚至优于所有之前报告的集成模型。

我们对基于注意力的模型的未来充满期待，并计划将它们应用于其他任务。我们计划扩展 Transformer 模型，以处理除文本以外的其他输入和输出模态的问题，并研究局部、受限的注意力机制，以高效地处理大型输入和输出，如图像、音频和视频。使生成过程不再严格依赖于序列顺序也是我们的研究目标之一。

我们用于训练和评估模型的代码可在 https://github.com/tensorflow/tensor2tensor 上获取。

致谢 我们感谢 Nal Kalchbrenner 和 Stephan Gouws 提供的富有成效的评论、修正和灵感。

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.

[8] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.

[9] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[11] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[13] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

[14] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

[15] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.

[16] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.

[17] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[18] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.

[19] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[20] Samy Bengio Łukasz Kaiser. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

参考文献

[1] Jimmy Lei Ba, Jamie Ryan Kiros, 和 Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, 和 Yoshua Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, 和 Quoc V. Le. Massive exploration of neural machine translation architectures. CoRR, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, 和 Mirella Lapata. Long short-term memory-networks for machine reading. arXiv preprint arXiv:1601.06733, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.

[8] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. arXiv preprint arXiv:1705.03122v2, 2017.

[9] Alex Graves. 用循环神经网络生成序列。arXiv preprint arXiv:1308.0850, 2013.

[10] 李航, 张翔宇, 任少清, 和 孙剑. 深度残差学习用于图像识别
年龄识别。见于 IEEE 计算机视觉与模式识别会议论文集，第 770–778 页，2016 年。

[11] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, 和 Jürgen Schmidhuber. 反向传播在递归网络中的梯度流：学习长期依赖关系的困难，2001 年。

[12] Sepp Hochreiter 和 Jürgen Schmidhuber. 长短期记忆. 神经计算, 9(8):1735–1780, 1997.

[13] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, 和 Yonghui Wu. 探索语言建模的极限. arXiv preprint arXiv:1602.02410, 2016.

[14] Łukasz Kaiser 和 Ilya Sutskever. 神经 GPU 学习算法. 在国际学习表征会议 (ICLR) 上, 2016.

[15] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, 和 Koray Kavukcuoglu. 线性时间的神经机器翻译. arXiv preprint arXiv:1610.10099v2, 2017.

[16] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. 结构化注意力网络.
在国际学习表征会议, 2017.

[17] Diederik Kingma 和 Jimmy Ba. Adam: 一种随机优化方法. 在 ICLR, 2015.

[18] Oleksii Kuchaiev 和 Boris Ginsburg. LSTM 网络的因子化技巧. arXiv preprint arXiv:1703.10722, 2017.

[19] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, 和 Yoshua Bengio. 一种结构化的自注意力句子嵌入. arXiv preprint arXiv:1703.03130, 2017.

[20] Samy Bengio Łukasz Kaiser. 激活记忆能否替代注意力? 在 Advances in Neural Information Processing Systems (NIPS) 中, 2016.

[21] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

[22] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.

[23] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

[24] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[25] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[26] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[27] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[28] Sainbayar Sukhbaatar, arthur szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.

[29] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.

[30] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.

[31] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[32] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

[21] Minh-Thang Luong, Hieu Pham, 和 Christopher D Manning. 基于注意力的神经机器翻译的有效方法。arXiv preprint arXiv:1508.04025, 2015.

[22] Ankur Parikh, Oscar Täckström, Dipanjan Das, 和 Jakob Uszkoreit. 一种可分解的注意力模型。在 Empirical Methods in Natural Language Processing, 2016.

[23] Romain Paulus, Caiming Xiong, 和 Richard Socher. 用于摘要生成的深度强化模型。arXiv preprint arXiv:1705.04304, 2017.

[24] Ofir Press 和 Lior Wolf. 利用输出嵌入来改进语言模型。arXiv preprint arXiv:1608.05859, 2016.

[25] Rico Sennrich, Barry Haddow, 和 Alexandra Birch. 用子词单元进行稀有词的神经机器翻译。arXiv 预印本 arXiv:1508.07909, 2015.

[26] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, 和 Jeff Dean. 极其庞大的神经网络：稀疏门控的专家混合层。arXiv 预印本 arXiv:1701.06538, 2017.

[27] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, 和 Ruslan Salakhutdi- 11 月 Dropout: 一种防止神经网络过拟合的简单方法。机器学习研究杂志，15(1):1929–1958, 2014.

[28] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. 端到端记忆网络。由 C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama 和 R. Garnett 编辑，神经信息处理系统进展 28, 第 2440–2448 页。Curran Associates, Inc., 2015.

[29] Ilya Sutskever, Oriol Vinyals, 和 Quoc VV Le. 基于神经网络的序列到序列学习。见 Advances in Neural Information Processing Systems，第 3104–3112 页，2014 年。

[30] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, 和 Zbigniew Wojna. 重新思考用于计算机视觉的 inception 架构。CoRR, abs/1512.00567, 2015.

[31] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, 等. Google 的神经机器翻译系统：弥合人类与机器翻译之间的差距. arXiv preprint arXiv:1609.08144, 2016.

[32] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, 和 Wei Xu. 用于神经机器翻译的具有快速前向连接的深度循环模型. CoRR, abs/1606.04199, 2016.